

An Architectural Framework for Formal Data Modeling in TiddlyWiki: A “Schema-on-Validate” Approach for Large-Scale Refactoring

Wulf Kolbe
Gemini 2.5
Kimi, Moonshot AI

November 16, 2025

Abstract

This report presents a formal architectural blueprint for a data management framework within TiddlyWiki, analogous to an Object-Relational Mapper (ORM), designed to facilitate large-scale, LLM-driven code refactoring projects. The framework introduces a hybrid “Schema-on-Validate” layer that reconciles traditional “schema-on-write” paradigms with TiddlyWiki’s native “schema-on-read” architecture. By establishing formal *Entity* definitions using the Zod validation library, the framework provides explicit, machine-readable schemas that serve as a single source of truth for data validation, static type generation, and LLM instruction. The report details the complete methodology from schema discovery and definition syntax to implementation of ORM functions and advanced templated transclusion patterns, providing a comprehensive roadmap for refactoring complex TiddlyWiki applications.

1 Part 1: A Formal Data-Definition Framework for TiddlyWiki

This report presents a formal architectural blueprint for a data management framework within TiddlyWiki, analogous to an Object-Relational Mapper (ORM), designed to facilitate a large-scale, LLM-driven code refactoring project.

The central challenge is to reconcile the “schema-on-write” paradigm of traditional ORMs with TiddlyWiki’s native “schema-on-read” architecture. TiddlyWiki, at its core, is a flexible key-value store where structure is not enforced at the point of data creation but rather applied at the point of query and presentation. The “schema” is implicit, existing only within the TiddlyWiki Filter expressions and rendering templates that consume the data.

While this flexibility is a core feature, it creates significant challenges when scaling, as noted in the project context of 10 applications and over 200 code modules. This “implicit definition by example” leads to “fragmented” logic and “implicit coordination”, where a change in one module’s data assumption can silently break numerous others. This is an

untenable state for a large-scale, LLM-assisted refactoring, which requires explicit, “clear definitions” as a source of truth.

The objective is not to fundamentally alter TiddlyWiki’s core or to force a rigid “schema-on-write” model, which would be counter-productive. Instead, this report architects a hybrid “**Schema-on-Validate**” layer. This layer is a set of formal definitions and tools that overlay the core system. It provides the explicit “language” and “restrictions” required for a robust system, without sacrificing the underlying flexibility.

This framework applies a formal, explicit schema at discrete, high-leverage points in the data lifecycle:

1. **At Definition:** Establishing an explicit, machine-readable “language” for data structures using modern schema validation libraries.
2. **At Creation:** Employing validation-aware “constructors” (customized TiddlyWiki widgets) to create new, schema-compliant data tiddlers.[?]
3. **At Query-Time:** Continuing to use TiddlyWiki’s powerful filter language as the primary data query mechanism.[?]
4. **At Presentation:** Engineering “smart” templates that validate the data they receive before rendering.[?]
5. **At Refactoring:** Providing the explicit schemas and examples as the primary instruction set for a Large Language Model.

This report will architect this framework, moving from (1) formal terminology, to (2) the definition syntax, (3) the implementation of ORM functions, (4) the advanced transclusion case, and (5) the final methodology for schema discovery and LLM-driven refactoring.

2 Part 2: Defining the Terminology: TiddlyWiki “Collections” and “Entities”

The request for a formal name for “tables” is the foundational step in creating this new data language. The terminology of relational databases (tables, rows, columns) is a poor fit for the NoSQL-like, non-relational model of TiddlyWiki.

2.1 The “Table” in NoSQL

In NoSQL databases, the concept of a “table” is replaced by other, more flexible constructs:

- **Document Databases:** Systems like MongoDB and CouchDB use the term “**Collection**” to group a set of “**Documents**” (the equivalent of rows).
- **Graph Databases:** Systems like Neo4j use “**Nodes**” (entities) and “**Edges**” (relationships) as their primary structures.

The TiddlyWiki-as-Document-DB model is the most accurate analogy. In this model:

- A TiddlyWiki HTML file is a single **Database**.
- This database contains one massive, undifferentiated “**Collection**” of tiddlers.
- Each tiddler is a “**Document**”.
- Each tiddler field is a “**Field**”.

2.2 Solving the “Single Collection” Problem

The primary challenge is that all data “types” (e.g., “BibKey,” “JournalEntry,” “Task”) coexist within this single, global collection of tiddlers. This is a classic modeling problem in document databases. The established best practice for segregating data *within* a single collection is to add a “type” or “entity” field to the document itself, which acts as a discriminator.[?] This allows queries to filter for only the documents of a specific type.

TiddlyWiki’s existing “definition by example” (e.g., “all data sets that has a tag beginning with ‘bigkey’ belongs to the same ‘table-like’ structure”) is a native implementation of this exact pattern. This “type” discriminator in TiddlyWiki is not necessarily a field named type (which has a specific meaning in TiddlyWiki for content-type), but is more commonly implemented via:

1. **Tags:** A filter such as [tag[BibKey]].
2. **Title Prefixes:** A filter such as [prefix[BibKey_]].
3. **A Custom Field:** A filter such as [entity[BibKey]].

2.3 Proposed Formal Terminology

To establish a clear language for the new framework, the following terminology will be adopted:

- **Entity:** The abstract, formal *definition* of a data structure. This is the “language” to be developed. (e.g., “The BibKey Entity requires a title, author, and year field.”)
- **Collection:** The *set* of tiddlers (documents) that conform to a specific Entity definition. A Collection is defined by the TiddlyWiki filter that selects its members. (e.g., “The BibKey Collection is defined as all tiddlers matching the filter [prefix[BibKey_]].”)
- **Instance:** A single tiddler (document) that is a member of a Collection.

This terminology (Entity, Collection, Instance) is precise, maps directly to NoSQL concepts, and formalizes the existing, implicit TiddlyWiki practices.

3 Part 3: The Core Paradigm: A “Schema-on-Validate” Architecture

TiddlyWiki’s native data model is a “schema-on-read” system. This means the system applies a schema or structure *during* data analysis (querying), not *before* data storage. Data is ingested in its raw form, and structure is imposed by the TDB ListWidget filters[?], transclusion templates, and ViewWidget. The “schema” is implicit in the code that *consumes* the data.

3.1 The Problem with Implicit Schemas

As a system scales, this implicit “schema-in-the-code” becomes a significant liability. It leads to “fragmented” logic and “implicit coordination,” which are high-risk anti-patterns in software development. When the implicit schema for a BibKey tiddler is defined slightly differently across 10 applications and 200 modules, the system becomes brittle and impossible to refactor. A change to a field name in one application’s template can silently break all others.

This is precisely the state that makes automated, LLM-driven refactoring unfeasible. The LLM has no “source of truth” to consult. It cannot determine if author or authors is the correct field, or if the year field is optional, required, or has a default.

3.2 The “Schema-on-Validate” Layer

The architectural solution is to create a “**Schema-on-Validate**” layer. This layer does not replace TiddlyWiki’s flexible “schema-on-read” core; it overlays it with a set of explicit definitions and validation tools.

This hybrid approach directly satisfies the need for a system that is “not strict but [imposes] restrictions.” A tiddler *can* still be saved without conforming to its Entity schema, preserving TiddlyWiki’s core flexibility. However, the “Schema-on-Validate” framework provides the tools to *detect*, *flag*, *validate*, and *manage* these tiddlers at key points in their lifecycle.

The implementation of this layer consists of:

1. **The Schema Definition:** A set of tiddlers (e.g., application/javascript modules) that explicitly define the *Entities* using a formal schema language.
2. **Validation Points:**
 - **On Create:** A custom “New” button (using `tm-new-tiddler`) that consults the Entity schema to prepopulate a new Instance with the correct fields and default values.
 - **On View/Transclude:** “Schema-Aware Templates” (discussed in Part 6) that validate an Instance *before* rendering it, and display an error if required fields are missing.
 - **On Migrate:** Batch-processing tools (discussed in Part 5) that use the Entity schema to *find* non-compliant Instances and *transform* them into the new, correct structure.
 - **On LLM Refactor:** The explicit Entity schema itself becomes the primary “prompt” and “instruction set” for the LLM.

4 Part 4: The Syntax: A Data Definition Language (DDL) for TiddlyWiki Entities

This section delivers the “syntax to define such structures.” A formal, explicit, and machine-readable language is required. While several candidates exist, the choice has a profound impact on the primary goal of LLM-driven refactoring.

4.1 Evaluating Schema Languages

- **JSON Schema:** A robust, standard vocabulary for defining JSON data structures. It clearly defines type, properties, required, and default.[?] These schemas could be stored in TiddlyWiki Data Tiddlers.[?]
- **Joi:** A popular and powerful, fluent API-based validation library.
- **Zod:** A modern, TypeScript-first schema validation library. It provides a concise interface for defining schemas, including complex rules via preprocess, refine, optional, and default.

4.2 Why Zod is the Architecturally-Superior Choice

The ultimate goal is to refactor 200+ *code modules*. A static JSON Schema defines *data*. Zod defines *data* and simultaneously infers the *static TypeScript type* that represents that data in code.

This `z.infer` capability is the critical architectural advantage. By defining an Entity in Zod, the framework gains:

1. **A Runtime Validator:** The schema can be imported into TiddlyWiki JavaScript modules to parse and validate tiddler fields at runtime.
2. **A Static Type Generator:** The LLM can infer the static type (e.g., `type BibKey = z.infer<type>`) directly from the schema. This allows the LLM to understand the *exact* object shape the code modules are intended to manipulate.
3. **A Single Source of Truth:** The schema *is* the type. This “eliminate[s] the need for duplicative type declarations” and directly prevents the “drift between template and code” that plagues large, schema-less projects.

Zod, therefore, is not just a validator; it is the *bridge* between the TiddlyWiki data (string-based fields) and the JavaScript code (modules) that manipulates them.

4.3 Proposed Implementation: Zod Entity Definitions

For each Entity, a JavaScript module tiddler (e.g., `$/schemas/BibKey.js`) will be created. This tiddler will be tagged `$/tags/TiddlyWiki/Module` and will export its Zod schema.

4.3.1 Example Syntax for a BibKey Entity:

Listing 1: BibKey Entity Schema Definition

```

/* Tiddler: $:/schemas/BibKey.js */
/* type: application/javascript */
/* module-type: library */

const { z } = require("$:/plugins/path-to-zod/zod.js");

/*
 * This is the formal Entity definition for a BibKey Instance.
 * It provides runtime validation and static type inference.
 */
exports.BibKeySchema = z.object({

  // --- Tiddler Identity Fields (from discriminator pattern) ---
  title: z.string()
    .startsWith("BibKey_", "Title must start with 'BibKey_'")
    .refine(val => val.length > 7, "Title must not be empty (e.g., 'BibKey_ID')"),

  tags: z.string() // TW 'tags' field is a space-delimited string
    .transform(val => val.split(' ')) // Preprocess string to string
    .pipe(z.array(z.string())) // S34
    .refine(tags => tags.includes("bibkey.BibKey"), {
      message: "Tiddler must be tagged 'bibkey.BibKey'"
    }),

  // --- Custom Data Fields (User-Defined Schema) ---
  author: z.string({
    required_error: "Author field is required.",
    invalid_type_error: "Author must be a string."
  }).min(3, "Author name is too short."),

  year: z.preprocess( // S34
    (val) => String(val).replace(/\D/g, ''), // Clean non-numeric chars
    z.string().length(4, "Year must be 4 digits")
    .or(z.string().length(0)) // Allow empty string
  ).optional(), // This field is not required

  journal: z.string().optional(),

  // --- Handling defaults for non-existence ---
  status: z.string().default("pending"), // S201

  // --- Standard TiddlyWiki Fields (for completeness) ---
  // All TW fields are strings, even dates
  created: z.string().optional(),
  modified: z.string().optional(),

}).strict(); // Disallow any fields *not* defined in this schema.

/*
 * This is the static type that will be used by the LLM
 * and in all JavaScript modules.
 */

```

```
* Example:
*   type BibKey = z.infer<typeof exports.BibKeySchema>;
*
*   const result = BibKeySchema.safeParse(tiddler.fields);
*   if (result.success) {
*       const bibKeyData: BibKey = result.data;
*       // bibKeyData.author is now a typesafe, validated string
*       // bibKeyData.status is "pending" even if field was missing
*   }
*/
```

4.4 Table: Entity Definition Syntax Comparison

This table provides a “Rosetta Stone” that translates from the current “implicit” state to the proposed “explicit” Zod-based language, demonstrating its value for LLM instruction.

Table 1: Entity Definition Syntax Comparison

Feature	Implicit (TiddlyWiki “by example”)	Explicit (JSON Schema)	Explicit (Zod)
“Table” Collection	Tag: bibkey.BibKey, Title: BibKey_	properties: { tags: { type: "string" },... }	tags: z.string()....refine(..., "must have tag"), title: z.string().startsWith(...)
Field Definition	Tiddler has a field author	properties: { author: { "type": "string" } }	author: z.string()
Required Field	The FO template breaks if author is missing.	"required": ["author"]	author: z.string() (non-optional by default)
Optional Field	The year field is sometimes present.	properties: { year: { "type": "string" } }	year: z.string().optional()
Default Value	If status is missing, <i>code assumes</i> it’s “pending”.	properties: { status: { "type": "string", "default": "pending" } } }[?]	status: z.string().default("pending")
Type for LLM/Code	any (a “bag of strings”)	type: "object"	type BibKey = z.infer<...>

5 Part 5: Implementing ORM Functions (DML) in TiddlyWiki

This section provides the implementation details for the requested “basic functions,” creating a TiddlyWiki-native Data Manipulation Language (DML) that operates on the Entities defined in Part 4.

5.1 Define Structures

This function is accomplished by creating the Zod Entity schema tiddlers as detailed in Part 4.

5.2 Initiate Structures / New Object (Seeding)

This is the ORM's "constructor" or "new Object" function.

- **Mechanism:** The WidgetMessage: `tm-new-tiddler[?]`
- **Analysis:** This message is TiddlyWiki's native "new" operator. It can be parameterized to use a template tiddler (`$param`) and, most importantly, *pass in additional fields* to prepopulate the new tiddler.
- **Implementation:** A button that creates a new BibKey Instance, enforcing the schema's defaults and required fields upon creation.

Listing 2: New BibKey Entry Button

```
<$button>
  <$action-sendmessage
    $message="tm-new-tiddler"
    $param="$:/templates/BibKeyTemplate"
    title="BibKey_NewEntry"
    tags="bibkey.BibKey"
    status="pending"
    author=""
    year=""
    journal=""
  />
  New BibKey Entry
</$button>
```

This action creates a new draft tiddler with all the fields the BibKey Entity expects, ready for the user to fill. The `status="pending"` is explicitly set, matching the default("pending") in the Zod schema.

5.3 Creating Data Sets (Insert / Update)

This is the ORM's INSERT and UPDATE function.

- **Mechanism:** The ActionSetFieldWidget[?]
- **Analysis:** This widget is the primary DML operator for modifying data. It can set one or multiple fields on a target tiddler. If the tiddler does not exist, it will be created.
- **Implementation (Example: updating a status):**

Listing 3: Update Status Button

```
<$button>
  <$action-setfield $tiddler=<<currentTiddler>> status="complete" />
  Mark as Complete
</$button>
```

5.4 Delete Data Sets

This is the ORM's DELETE function.

- **Mechanism:** The ActionDeleteTiddlerWidget[?]
- **Analysis:** This widget can delete a single tiddler (via the \$tiddler attribute) or, more powerfully, delete an entire set of tiddlers that match a filter (via the \$filter attribute).[?]
- **Implementation (Example: delete all pending BibKey entries):**

Listing 4: Delete All Pending Button

```
<$button actions='<$action-deletetiddler $filter="[status[pending]
  prefix[BibKey_]]"/>'>
  DELETE ALL PENDING
</$button>
```

5.5 Search for Data Sets (Querying)

This is the ORM's find or SELECT function.

- **Mechanism:** TiddlyWiki Filter Syntax[?]
- **Analysis:** The filter *is* the query. This is the direct mapping from SQL/ORM query logic to TiddlyWiki's DML.
- **Examples:**
 - SELECT * FROM BibKey → [prefix[BibKey_]]
 - SELECT * FROM BibKey WHERE year = '2024' → [prefix[BibKey_]year[2024]]
 - SELECT * FROM BibKey WHERE journal IS NOT NULL → [prefix[BibKey_]has[journal]]
 - SELECT COUNT(*) FROM BibKey WHERE status = 'pending' → [prefix[BibKey_]status[pending]]

5.6 Migrate Data to New Structures

This directly addresses the “search, modify, insert” compound action.

- **Mechanism:** A ListWidget (for search) wrapping ActionSetFieldWidget (for modify/insert)[?]
- **Analysis:** This is TiddlyWiki's batch processing model. The ListWidget iterates over every tiddler that matches the “search” filter. For each tiddler, it executes the ActionWidgets nested inside it. The ActionSetFieldWidget serves as the “modify” (updating a field) and “insert” (creating a new field) operator.
- **Implementation (Example: rename field author to creator across all BibKey Instances):**

Listing 5: Migrate Author Field to Creator

```
<$button>
  <$list filter="[prefix[BibKey_]has[author]]">
    <$action-setfield $tiddler=<<currentTiddler>> creator={{!!
      author}} />
    <$action-deletefield $tiddler=<<currentTiddler>> author />
  </$list>
  Migrate Author Field to Creator
</$button>
```

5.7 Organize, Show Tables, Show Statistics

These functions relate to data presentation and aggregation.

- **Show Tables (Collections):** A “table” is simply a ListWidget that displays all Instances of an Entity, typically using a template.[?]
- **Show Statistics:** This is accomplished via dashboarding and visualization plugins (e.g., ECharts, vis.js). The ECharts plugin can be fed data *directly from a filter*.[?] This allows for the creation of powerful, reactive dashboards that can, for example, display a bar chart of BibKey Instances grouped by year, with the data source being a TiddlyWiki filter.

5.8 Table: ORM Function to TiddlyWiki Implementation Map

This table serves as a comprehensive “cookbook” for the DML, mapping ORM concepts directly to TiddlyWiki primitives.

6 Part 6: Advanced Implementation: Validated Templated Transclusion

This section provides a direct, technical solution for the templated transclusion use case: BibKey_FO0007||FOBibKey_FO0007||FO.

6.1 The Transclusion Mechanism

This WikiText is a TiddlyWiki shorthand for a “Transclusion with Template”. As confirmed by TiddlyWiki documentation, the syntax Target||TemplateTarget||Template (e.g., BibKey_FO0007||FOBibKey_FO0007||FO) is functionally equivalent to:[?]

Listing 6: Transclusion Mechanism

```
<$tiddler tiddler="BibKey_F00007">
  <$transclude $tiddler="FO"/>
</$tiddler>
```

The outer <\$tiddler> widget sets the currentTiddler variable to BibKey_FO0007. The inner <\$transclude> widget then renders the FO tiddler *within that context*.[?] Any generic field reference inside FO (e.g., !!author !!author) will pull data from BibKey_FO0007.

Table 2: ORM Function to TiddlyWiki Implementation Map

ORM Function	Function	TiddlyWiki Mechanism	Example WikiText / Syntax	Sources
Define Structure (DDL)	Structure	JS Module + Zod Schema	.exports.MySchema = z.object(	S32, S76, S201
New Object (Seed)	Object	tm-new-tiddler Message	<\$action-sendmessage \$message="tm-new-tiddler" \$para	S135, [?]
Insert / Update (DML)	Update	ActionSetFieldWidget	<\$action-setfield \$tiddler="Title" \$field="value" />	S149, [?]
Delete (DML)		ActionDeleteTiddlerWidget	<\$action-deletetiddler \$tiddler="Title" or \$filter = "..."	S148, [?]
Search (All)		Filter by Tag/Prefix	[tag[X]] or [prefix [X]]	S68, S191
Search (by Field)	(by Field)	Filter by Field Value	[year[2024]]	S145, S212
Search (Field Exists)	(Field Exists)	has Filter Operator	[has[journal]]	S143, S218
Aggregate (Count)		count Filter Operator	[count []]	S211
Migration (Batch)		ListWidget + ActionWidgets	<\$list filter = "[...search...]"><\$action-setfield ... modify.	S150, [?]
Display Table		ListWidget	<\$list filter = "[...]" template="MyTemplate"/>	S152, [?]
Display Statistics		ECharts Plugin + Filter	<\$echarts \$tiddler="MyFilterData" />	S144, [?]

6.2 The Validation Problem

The requirement is that “a template FO should only access Tiddler that have the needed fields.” The FO template must be “smart” and *validate* that the currentTiddler (which has been passed to it) actually *has* the fields it expects (e.g., author, year) before it attempts to render them.

6.3 The Solution: The “Schema-Aware” Template

This validation can be performed *inside* the FO template using the ListWidget as a conditional (IF) block.

The ListWidget’s content is only rendered if its filter attribute returns one or more items.[?] We can use the has filter operator to test the currentTiddler. The filter [all [current] has[author] will *only* return the currentTiddler if it possesses *both* the author and year fields. If it is missing either, the filter returns an empty list.

This allows for the creation of a “Schema-Aware Template”:

6.3.1 Example FO Template Tiddler:

Listing 7: Schema-Aware Template

```
<$list
```

```

filter="[all[current]has[author]has[title]]"
emptyMessage="<$span style='color:red; font-weight:bold;'>
    ERROR: Tiddler does not have required fields for FO template.
</$span>"
>
<span class="bibkey-reference">
    (<$link to={{!!title}}><$text text={{!!author}}/>,

    <$list filter="[all[current]has[year]]" variable="null">
        <$text text={{!!year}}/>
    </$list>

    )
</span>

<$list filter="[all[current]has[journal]]">
    <span class="bibkey-journal">
        <i>In: <$text text={{!!journal}}/></i>
    </span>
</$list>

</$list>

```

This pattern perfectly achieves the “not strict but imposing restrictions” goal. The “broken” tiddler (BibKey_FO0007) is not prevented from being saved, but at the point of presentation (transclusion), its non-compliance is gracefully caught and reported. This is the “Schema-on-Validate” pattern applied at the presentation layer.

7 Part 7: A Methodology for Schema Discovery and LLM Refactoring

This final section provides a strategic roadmap for the 10-app, 200-module refactoring project, synthesizing all previous parts into an actionable plan. The goal is to “bring the implicit definition... to a language” to provide “clear definitions” for an LLM.

7.1 Phase 1: Schema Discovery (From “Example” to “Language”)

The 10 applications already contain an “implicit schema”. This schema must be discovered and made explicit. It is not feasible to write this from scratch.

1. **Export:** For each of the 10 applications, generate a JSON export of all relevant tiddlers (e.g., all tiddlers tagged bibkey.BibKey). TiddlyWiki’s export mechanisms can facilitate this.
2. **Infer:** Use an automated schema discovery tool (such as JSONDiscoverer[?] or an online “JSON to JSON Schema” converter[?]) to analyze the exported JSON. These tools “infer... the implicit schema”[?] by analyzing the provided “definition by example” data.
3. **Consolidate:** This process will yield 10 *different* inferred JSON schemas (one for each application). A critical, human-led architectural step is to merge these 10 schemas into one “master” JSON schema that accounts for all variations (e.g., one

app uses author, another uses authors; one app’s year is a string, another’s is a number).

7.2 Phase 2: Schema Refinement (From JSON Schema to Zod Entity)

This phase translates the static JSON schema into the “live” Zod Entity definitions.

1. **Translate:** Manually, or with LLM assistance, translate the “master” JSON schema from Phase 1 into the far more powerful Zod Entity definitions detailed in Part 4.
2. **Enrich:** This is the most critical human-in-the-loop step. The architect must enrich the auto-generated schema with the semantic business logic that the discovery tool *cannot* infer:
 - Add semantic default () values (e.g., status: "pending").
 - Add preprocess() logic for data cleaning (e.g., standardizing all year fields).
 - Add refine () rules for complex validation (e.g., enforcing title prefixes or required tags).
 - **Critically:** Add descriptive comments and description attributes.

7.3 Phase 3: LLM Refactoring (“The LLM Function Design Pattern”)

This phase executes the refactoring of the 200+ code modules.

- **Schemas-as-Instructions for LLMs:** The “Schema-on-Validate” framework is designed to feed an LLM. The primary challenge in LLM-driven development is “fragmented” logic and “implicit coordination”. The Zod schemas solve this. As noted in analyses of LLM-JSON interactions, the description fields within a schema are *not* documentation for humans; they are “**potent instructions**” for the LLM.
- **The “Prompt Package” for the LLM:** To refactor a single module, the LLM must be provided with a complete “Prompt Package” containing:
 1. **The “Language” (Zod Schemas):** The full text of the relevant `$/schemas/*.js` module tiddlers. These *must* be heavily annotated with comments and descriptions that explain the *semantic intent* and *legacy conditions* (e.g., `// 'author' field is legacy`).
 2. **The “Data Context” (Examples):** A small set of JSON examples of “good” (schema-compliant) tiddlers and “bad” (legacy, non-compliant) tiddlers.
 3. **The “Task” (Code + Instructions):**
 - The original code from the module to be refactored.
 - A clear, high-level instruction prompt:

“You are an expert TiddlyWiki refactoring assistant. Your task is to refactor the provided TiddlyWiki JavaScript module.

 - (a) Import the provided Zod Entity Schemas (e.g., `const { BibKeySchema } = require`
 - (b) Identify all code that *implicitly* accesses tiddler fields (e.g., `var author = tiddler . fi`

- (c) Replace all implicit access with an *explicit, validated access pattern*.
- (d) Before accessing data, parse the tiddler’s fields using the schema:
`const result = BibKeySchema.safeParse(tiddler.fields);`
- (e) Check the `result.success` flag. If false, handle the error gracefully (e.g., log a warning, return an error message).
- (f) If true, use the validated, cleaned, and defaulted data from the result: `var author = result.data.author;`
- (g) Ensure all ListWidget filters are updated to use the new, canonical field names defined in the schema.”

This methodology directly leverages the explicit “language” (the Zod Entities) to solve the “implicit schema” problem, providing the LLM with the “clear definitions” it needs to perform the large-scale refactoring reliably and consistently.

8 Part 8: Conclusions and Architectural Recommendations

The analysis and synthesis of the query and associated data lead to the following conclusions:

1. **The “Schema-on-Validate” Framework:** A hybrid architecture that overlays a “Schema-on-Validate” layer on TiddlyWiki’s native “schema-on-read” engine is the optimal solution. This approach provides the “restrictions” and “clear definitions” necessary for a large-scale project without sacrificing TiddlyWiki’s core flexibility.
2. **A Formal Language:** This report has established a formal language for this framework.
 - **Terminology:** Data structures are **Entities**, queries are **Collections**, and tiddlers are **Instances**.`[?]`
 - **DDL (Syntax):** The **Zod** schema library is the recommended technology for defining Entities. Its ability to infer static types (`z.infer`) makes it architecturally superior to JSON Schema for the stated goal of LLM-driven code refactoring.
 - **DML (Functions):** TiddlyWiki’s core primitives (`ActionWidgets[?]`, `Filter Operators`, and the `ListWidget[?]`) provide a complete, robust DML for all requested ORM functions (`Create`, `Search`, `Delete`, `Migrate`).
3. **Problem Resolution:** This framework provides direct solutions to all stated requirements:
 - It provides a “syntax to define structures” (Part 4).
 - It handles “defaults for none existence” (via `Zod’s default()`).
 - It provides a “Schema-Aware Template” pattern to solve the “templated transclusion” problem (Part 6).

- It provides a “language” to “bring the implicit definition... by example” into an explicit, formal system (Part 7).
4. **Final Recommendation:** The success of the 200-module refactoring project hinges entirely on the human-led **Phase 2: Schema Refinement**. The automated discovery (Phase 1) will provide a baseline, but the true value is in the human-authored *enrichment* of the Zod schemas. The quality of the preprocess, refine, and (most importantly) the descriptive comments in the Zod Entity definitions will directly determine the quality, accuracy, and success of the LLM’s refactoring output.

References

- [1] TiddlyWiki v5.3.8 — a non-linear personal web notebook, “WidgetMessage: tm-new-tiddler,” accessed November 16, 2025, <https://tiddlywiki.com/static/WidgetMessage%253A%2520tm-new-tiddler.html>
- [2] TiddlyWiki v5.3.8 — a non-linear personal web notebook, “Filters,” accessed November 16, 2025, <https://tiddlywiki.com/static/Filters.html>
- [3] “Using fields as templates - Talk TW,” accessed November 16, 2025, <https://talk.tiddlywiki.org/t/using-fields-as-templates/6268>
- [4] “Best practice about Table Name \& Document ID in NoSQL ...,” accessed November 16, 2025, <https://www.couchbase.com/forums/t/best-practice-about-table-name-document-id-in-nosql/4510>
- [5] “ListWidget - TiddlyWiki,” accessed November 16, 2025, <https://tiddlywiki.com/static/ListWidget.html>
- [6] “Annotations - JSON Schema,” accessed November 16, 2025, <https://json-schema.org/understanding-json-schema/reference/annotations.html>
- [7] “Data Tiddlers: Grok TiddlyWiki — Build a deep, lasting ...,” accessed November 16, 2025, <https://groktiddlywiki.com/static/Data%2520Tiddlers.html>
- [8] TiddlyWiki v5.3.8 — a non-linear personal web notebook, “ActionSetFieldWidget,” accessed November 16, 2025, <https://tiddlywiki.com/static/ActionSetFieldWidget.html>
- [9] TiddlyWiki v5.3.8 — a non-linear personal web notebook, “ActionDeleteTiddlerWidget,” accessed November 16, 2025, <https://tiddlywiki.com/static/ActionDeleteTiddlerWidget.html>
- [10] “ECharts - Bringing amazing visualization tools to TiddlyWiki! - #19 ...,” accessed November 16, 2025, <https://talk.tiddlywiki.org/t/echarts-bringing-amazing-visualization-tools-to-tiddlywiki/1944/19>
- [11] TiddlyWiki v5.3.8 — a non-linear personal web notebook, “Transclusion in WikiText,” accessed November 16, 2025, <https://tiddlywiki.com/static/Transclusion%2520in%2520WikiText.html>

- [12] “JSONDiscoverer: Visualizing the schema lurking JSON documents,” accessed November 16, 2025, <https://modeling-languages.com/json-schema-discoverer/>
- [13] “Free Online JSON to JSON Schema Converter - Liquid Technologies,” accessed November 16, 2025, <https://www.liquid-technologies.com/online-json-to-schema-converter>